Software Engineering

INSTRUCTOR : DR. NATASH ALI MIAN

School of Computer and Information Technology

Beaconhouse National University

1 and 2

How many numbers are there between the above mentioned numbers?

Take input in a 2

Take input in b 3

Add

Display Result 5 (Expected Output)

Black Box (we consider the system as a black box)

White Box (Internal Analysis)

Software Testing

Software testing is a key activity, both in software development and in software maintenance. Its objectives are opposed to that of development: instead of making the system work, it aims at breaking it, thus revealing the presence of defects. This is the main reason why development and testing teams should be separate.

Code analyses are extremely helpful to testing. Specifically, structural testing (aka white-box testing), as opposed to functional testing (aka black-box testing), assumes the possibility to access the internal structure of the program, analyze it and derive testing criteria from such an analysis. The outcome supports:

- Test case production/automatic generation.
- Definition of stopping criteria.

Observations about Testing

“Testing is the process of executing a program with the intention of finding errors.” – Myers

“Testing can show the presence of bugs but never their absence.” - Dijkstra

Good Testing Practices

A good test case is one that has a high probability of detecting an undiscovered defect, not one that shows that the program works correctly

It is impossible to test your own program

A necessary part of every test case is a description of the expected result

Good Testing Practices (cont'd)

Avoid non reproducible or on-the-fly testing

Write test cases for valid as well as invalid input conditions.

Thoroughly inspect the results of each test

As the number of detected defects in a piece of software increases, the probability of the existence of more undetected defects also increases

Good Testing Practices (cont'd)

Assign your best people to testing

Ensure that testability is a key objective in your software design

Never alter the program to make testing easier

Testing, like almost every other activity, must start with objectives

Examples of test case

- Test Input Description:
  1. Login to <Abc page> as administrator
  2. Go to Reports page
  3. Click on the ‘Schedule reports' button
  4. Add reports
  5. Update
- Expected Results:
  The report schedule should get added to the report schedule Table

Test case for sort:

- Test data: <12 -29 32 >
- Expected output: -29 12 32

Terminology …

Failure: it is an observable incorrect behavior or state of a given system. In this case, the system displays a behavior that is contrary to its specifications/requirements. Thus, a failure is tied (only) to system executions/behaviors and it occurs at runtime when some part of the system enters an unexpected state.

Fault: (commonly named “bug/defect”) it is a defect in a system. A failure may be caused by the presence of one or more faults on a given system. However, the presence of a fault in a system may or may not lead to a failure, e.g., a system may contain a fault in its code but on a fragment of code that is never exercised so this kind of fault do not lead to a software failure.

Error: it is the developer mistake that produce a fault. Often, it has been caused by human activities such as the typing errors.

Example …

LOC	Code
1	program double ();
2	var x,y: integer;
3	begin
4	read(x);
5	y := x * x;
6	write(y)
7	end

Failure: x = 3 means y =9 →Failure!

- This is a failure of the system since the correct output would be *6*

Fault: The fault that causes the failure is in line 5. The * operator is used instead of *+*.

Error: The error that conduces to this fault may be:

- a typing error (the developer has written * instead of *+*)
- a conceptual error (e.g., the developer doesn't know how *to double a number*)

Test phases

Unit testing – this is testing of a single function, procedure, class. It is usually done by the developer, not by a separate testing team.

Integration testing – this checks that units tested in isolation work properly when put together. It often requires drivers and stubs to simulate the missing components while integrating the system.

System testing – here the goal is to ensure that the whole system works properly.

Acceptance Testing – this checks if the end user functionalities are actually delivered. It is often a contractual prerequisite for the user to accept and pay for the software.(UAT)

Regression Testing – this checks that the system preserves its functionality after maintenance and/or evolution.

Defect Seeding (testing/ Testing Team Efficiency)

Willful insertion of defects in to the system to check the efficiency of the testing team.

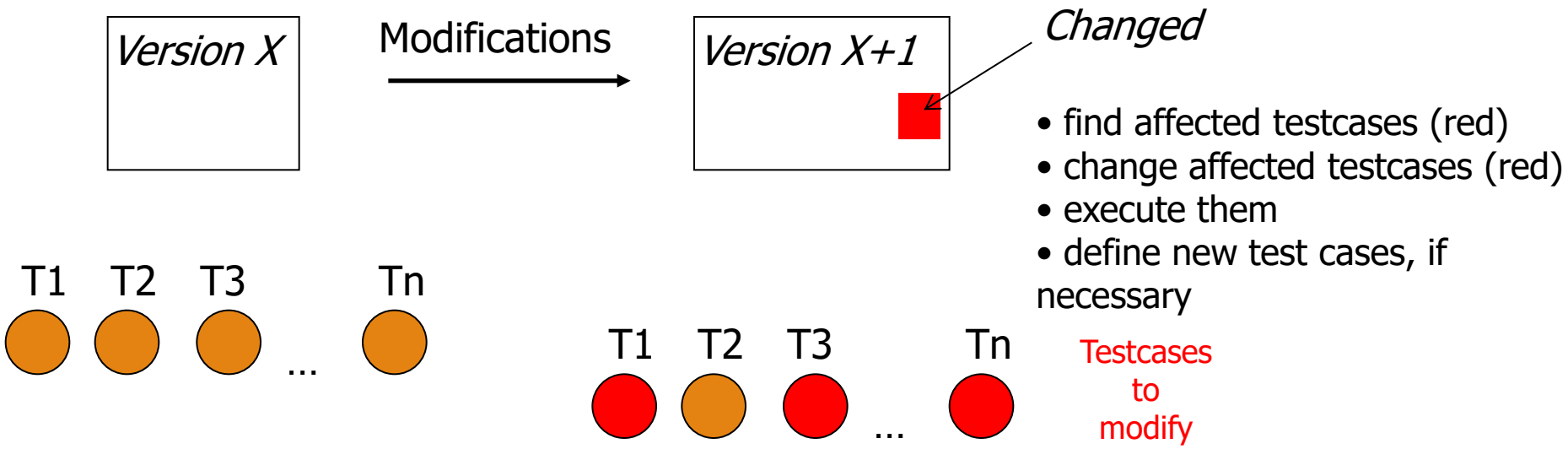
Generally, it is not liked by the testing team.

Regression testing

Selective re-testing of a system or component to verify that modifications have not caused unintended effects

- Ripple effects

Can be conducted at each of the test levels: unit, integration, system



Detailed Test Cases

The test cases will have a generic format as below.

- Test Case Id
- Test Case Description
- Test Prerequisite
- **Test Inputs**
- Test Steps
- **Expected Results**

Testing Vs Debugging

- Testing is focused on identifying the problems in the product
- Done by Tester
- Need not know the source code

- Debugging is to make sure that the bugs are removed or fixed
- Done by Developer
- Need to know the source Code